UNIVERSIDAD POLITÉCNICA METROPOLITANA DE HIDALGO

Maestría en Inteligencia Artificial

Algoritmo genético con tres codificaciones de cromosoma para la fusión de señales en un detector de fraude en reclamaciones médicas

Alumna	Jessica Melani Romero Lora
Matrícula	25322116
Catedrático	Dr. Jaime Aguilar Ortiz
Asignatura	Algoritmos Evolutivos
Periodo	Septiembre–diciembre de 2026
Fecha de entrega	21 de octubre de 2026

Código fuente:
https://github.com/jessicaromero-ctrl/Evolutive-Algorithms/tree/main/ACT-05-ALG-EVOL-CORTE-02-ROMERO-LORA-JESSICA-MELANI

1. Descripción del problema analizado con un algoritmo genético

1.1 Contexto industrial

Una unidad de investigación especial (SIU) de un asegurador de servicios médicos recibe cada periodo miles de reclamaciones de honorarios presentadas por proveedores. Cada reclamación se identifica con un procedimiento (código CPT/HCPCS), un diagnóstico (ICD-10), un monto facturado, un proveedor (NPI), una especialidad y fechas de servicio y de envío. La mayoría de las reclamaciones es legítima; una fracción pequeña es fraudulenta y se oculta entre las legítimas.

La restricción central es operativa: el equipo auditor puede revisar sólo una fracción fija de las reclamaciones, aquí el 5 % por periodo. El problema industrial no es clasificar bien en abstracto, sino ordenar las reclamaciones de modo que el 5 % revisado contenga la mayor cantidad posible de fraude, medida tanto en número de casos como en monto recuperable.

El detector descrito en los archivos de la materia combina tres capas de evidencia y dos clasificadores (un modelo de aprendizaje supervisado y un modelo no supervisado) en un sistema denominado O-A-R. El algoritmo genético de este estudio no detecta fraude por sí mismo: aprende cómo fusionar las seis señales que el detector produce, de modo que la puntuación final ordene mejor las reclamaciones.

1.2 Suposiciones de realismo industrial

Suposición	Justificación operativa	Límite
Fraude raro: 5.58 % de las reclamaciones en la línea base	El desequilibrio de clases es la regla en auditoría de salud; el área bajo la curva PR es la métrica adecuada	La tasa real no se calibró con datos de un asegurador
Revisión limitada al 5 % por pliegue	Capacidad fija del equipo auditor; las comparaciones se hacen a presupuesto idéntico	Un presupuesto distinto puede cambiar el ranking de métodos
Validación por proveedor	Un mismo proveedor no aparece en entrenamiento y prueba; evita fuga de su comportamiento	Los proveedores se asignan al azar a pliegues
Fraude con inicio tardío en la mitad de los proveedores fraudulentos	Las conductas aparecen en momentos distintos; genera deriva temporal real	El momento de inicio se fija por generador, no se observa en la realidad
Latencia de envío y horario de servicio distintos en fraude	Los fantasmas se envían rápido y en horas atípicas, es un supuesto de diseño del generador	Las probabilidades de esas señales son supuestos del generador
Fraude no adaptativo	El agente no cambia su conducta al detectar el sistema	No valida la componente de retroalimentación (CAS-4); es la limitación más importante
Etiquetas sólo en entrenamiento y evaluación	La inferencia de producción nunca ve la etiqueta; se audita automáticamente	Ninguna

Tabla 1. Suposiciones de realismo industrial del estudio

1.3 Datos sintéticos

El estudio se formula con suposiciones que replican condiciones operativas. Cada una se declara con su justificación y su límite, porque los resultados dependen de ellas. Los datos se generan con synpuf_generator_v2.py, inspirado en el archivo público SynPUF de Medicare Parte B (Centers for Medicare & Medicaid Services, s. f.).

Se generan 800 proveedores, 50,000 reclamaciones y 314 878 eventos de proceso, con semilla 2024. Los proveedores pertenecen a seis especialidades y siguen uno de seis perfiles de conducta. Los catálogos de procedimientos y tarifas son una versión simplificada de la base de datos Medicare Parte B; no reproducen tarifas reales.

Tabla 2. Perfiles de proveedor del generador

Perfil	Probabilidad	Mecanismo de fraude inyectado
normal	0.72	Ninguno
upcoder_leve	0.10	Facturar 99214 en lugar de 99213 con sobreprecio (F5)
upcoder_severo	0.07	Facturar 99215 o 99223 en lugar de consultas de menor nivel (F1)
phantom_biller	0.04	Procedimientos de catálogo sin evento clínico, con horario y envío atípicos (F2)

Perfil	Probabilidad	Mecanismo de fraude inyectado
unbundler	0.04	Monto inflado por desagregación de procedimientos (F3)
mixed_fraud	0.03	Mezcla de upcoding quirúrgico y severo (F4 y F1)

Probabilidades de cfg.profile_probs. Los fraudes sólo ocurren tras el inicio del proveedor.

Con dificultad 1.0 (línea base) el conjunto contiene 2,789 reclamaciones fraudulentas (5.58 %). El parámetro de dificultad $d \in (0, 1]$ escala la magnitud de las señales de fraude; con $d = 0.5$ y $d = 0.25$ se obtienen versiones más difíciles, usadas como prueba de estrés.

1.4 Detector base

El detector (cas_detector_v2.py) no usa etiquetas para construir variables. Sus señales son:

- **Capa A (estructural)** un modelo de Petri por especialidad, descubierto con el Inductive Miner (ruido 0.10) y evaluado con token-based replay. Su salida es la aptitud de traza (fitness) de cada reclamación.
- **Capa B (semántica)** soporte del par especialidad-procedimiento, monto relativo a la mediana del procedimiento y complejidad relativa al proveedor, con una cola superior de referencia.
- **Capa C (temporal)** función de distribución empírica bilateral del retraso de envío y de la hora de servicio.
- **XGBoost (dos variantes)** con variables estáticas, y con variables estáticas más las dimensiones CAS (deriva, entropía de códigos, cambios de régimen).
- **Isolation Forest** anomalía no supervisada sobre variables estáticas escaladas.
- **O-A-R** combinación por regla (OR o consenso de dos señales) y por promedio de rangos.

La validación es anidada por proveedor, con cinco pliegues. La auditoría de fuga confirmó que ninguna variable de entrada contiene el perfil ni la etiqueta (fuga: NO), y que todas las reclamaciones de prueba recibieron puntuación estructural (ninguna sin fitness). Verificamos además que los 800 proveedores caen en exactamente un pliegue.

2. Objetivo del estudio

Objetivo general. Diseñar, implementar y evaluar un algoritmo genético que aprenda los pesos de fusión de las seis señales del detector, comparando tres codificaciones del cromosoma, con validación por proveedor y presupuesto de revisión fijo.

Objetivos específicos.

- Formalizar el genotipo y el fenotipo del problema de fusión.
- Implementar tres codificaciones del cromosoma (binaria con código de Gray, real y entera), cada una con sus operadores.
- Comparar las codificaciones con dos controles de la misma familia: búsqueda aleatoria con presupuesto idéntico y pesos uniformes, más la mejor señal individual.

- Medir la calidad en prueba, la diversidad de la población, la velocidad de convergencia y el costo computacional.
- Evaluar la robustez en dos niveles de dificultad de los datos sintéticos.

Hipótesis. Se formularon antes de analizar los resultados del algoritmo genético; la última columna registra el veredicto.

Tabla 3. Hipótesis del estudio

Código	Hipótesis	Veredicto (dificultad 1.0)
H1	Al menos una codificación supera a pesos uniformes y a búsqueda aleatoria con el mismo presupuesto.	Se confirma (sección 7.3)
H2	Las codificaciones difieren en la diversidad de la población final (G2 > G1 > G3).	Se confirma (sección 7.4)
H3	Las codificaciones no difieren en calidad de prueba al presupuesto fijado.	Se confirma (sección 7.3)
H4	La fusión aprendida supera a la mejor señal individual.	Se confirma en significancia; magnitud despreciable (sección 7.3)
H5	La ventaja de la fusión sobre la mejor señal aumenta cuando la dificultad disminuye.	No se confirma (sección 7.6)

3. Las tres formas de codificar el cromosoma

Un cromosoma codifica una solución del problema, que aquí es un vector de pesos de fusión. Se definen tres codificaciones del mismo fenotipo, que difieren en la representación del genotipo y, por tanto, en los operadores que son naturales para cada una.

Tabla 4. Comparación de las tres codificaciones

	G1 · binaria (Gray)	G2 · real	G3 · entera
Genotipo	48 bits: 8 bits por cada uno de los 6 pesos	6 números reales en [0, 1]	6 enteros en {0, …, 20}
Tamaño del espacio	2^48 ≈ 2.8 × 10^14	Continuo (6 dimensiones)	21^6 = 85 766 121
Resolución del peso	1/255	Continua	1/20
Recombinación	Cruce de un punto sobre la cadena de bits	BLX-α con α = 0.3, recortado a [0, 1]	Cruce uniforme por gen
Mutación	Volteo de bit con probabilidad 1/48	Gaussiana acotada, σ = 0.05, probabilidad 1/6 por gen	Paso ±1 con probabilidad 1/6 por gen
Ventaja esperada	Operadores clásicos; exploración por bits	Precisión fina; relación directa con el fenotipo	Espacio finito; operadores simples

	G1 · binaria (Gray)	G2 · real	G3 · entera
Riesgo esperado	Epistasis: un bit cambia varios bits del peso	Redundancia por escala	Pérdida de diversidad al converger

La mutación binaria usa la tasa estándar 1/L, con L = 48 bits; las dos codificaciones por gen usan 1/K, con K = 6 genes.

La elección de tres codificaciones y no de una sola responde a una pregunta práctica: si la forma de representar los pesos altera la calidad o sólo el costo. Las tres codifican el mismo fenotipo y reciben las mismas poblaciones iniciales para una semilla dada, así que las diferencias observadas se atribuyen a la codificación y sus operadores, no a la muestra inicial.

4. Genotipos y fenotipos

4.1 Fenotipo común

El fenotipo es un vector de pesos en el símplex de dimensión K = 6, que corresponde a las seis señales del detector. La puntuación fusionada de la reclamación i es una combinación convexa de sus señales:

$$s_w(i) = \Sigma_k\, w_k \cdot z_ik\,, \quad w \in \Delta^5 = \{\, w \in \mathbb{R}^6 : w_k \geq 0,\ \Sigma_k\, w_k = 1 \,\} \qquad (1)$$

Las señales z_ik son s_struct, s_sem, s_temp, XGBoost con CAS, Isolation Forest y XGBoost sin CAS. Todas están en [0, 1] por construcción, así que no requieren normalización adicional.

4.2 Genotipo y función de decodificación

El genotipo G es la representación del cromosoma. La función φ lleva el genotipo a un vector intermedio u ∈ [0, 1]^6, y la normalización lleva u al símplex. Esta doble etapa separa la representación de la restricción de suma 1.

$$w(G) = u(G) \,/\, \Sigma_k\, u_k(G), \quad \text{con } w = (1/6, \ldots, 1/6) \text{ si } \Sigma_k\, u_k = 0 \qquad (2)$$

$$\text{G1:}\quad u_k = (1/255) \cdot \Sigma_{j=0}\hat{}\{7\}\, \hat{b}_\{kj\}\, 2\hat{}\{7-j\}, \quad \hat{b}_\{kj\} = \oplus_\{i \leq j\}\, g_\{ki\} \qquad (3a)$$

$$\text{G2:}\quad u_k = x_k\,, \quad x \in [0, 1]\hat{}6 \qquad (3b)$$

$$\text{G3:}\quad u_k = g_k \,/\, 20\,, \quad g_k \in \{0, 1, \ldots, 20\} \qquad (3c)$$

En G1, g_k,j son los bits del código de Gray del peso k, y $\hat{b}$_kj son los bits binarios recuperados con la suma XOR acumulada de izquierda a derecha. En G2 el genotipo es el fenotipo intermedio. En G3 cada gen toma uno de 21 valores equiespaciados.

4.3 Codificación inversa y redundancia

La población inicial se muestrea en u y se codifica con la inversa de (3a)–(3c). En G1, para un entero n = round(255 u_k), el gen se escribe como g = n ⊕ (n >> 1), de modo que enteros consecutivos difieren en un solo bit. Las pruebas automatizadas del código verifican esta propiedad.

Las tres codificaciones no son inyectivas respecto del fenotipo: por (2), u y c·u producen el mismo peso w para cualquier c > 0. Esta redundancia es mayor en G2 y G3, donde la escala de u es libre, La pérdida de diversidad de G3 (sección 7.4) se atribuye al espacio discreto, no a la redundancia de escala.

4.4 Función de aptitud

La aptitud de un genotipo es el área bajo la curva precisión–recall (PR-AUC, o precisión promedio) de la puntuación fusionada sobre las reclamaciones de los pliegues de entrenamiento. El área bajo la curva PR se prefiere a la ROC cuando la clase positiva es rara (Saito y Rehmsmeier, 2015).

$$f(G) = AP(w(G)) = (1 / P) \cdot \Sigma_{i : y_i = 1} \text{Prec@rank}(i), \quad P = \text{número de fraudes} \qquad (4)$$

Prec@rank(i) es la precisión sobre las reclamaciones ordenadas por s_w, hasta la posición de la i-ésima reclamación fraudulenta. La implementación vectorizada coincide con average_precision_score de scikit-learn cuando no hay empates (prueba de la función de aptitud).

5. Métodos usados en cada una de las siete etapas

Se usa el marco de siete etapas de la materia. Las etapas 1, 3, 6 y 7 son comunes a las tres codificaciones y se fijan; las etapas 4 y 5 dependen de la codificación. Los parámetros comunes son N = 60 individuos, torneo k = 2, elitismo e = 2, probabilidad de cruce p_c = 0.9 y presupuesto E_max = 5 000 evaluaciones de aptitud.

Tabla 5. Métodos por etapa y por codificación

Etapa	G1 binaria	G2 real	G3 entera
1 · Inicialización	Hipercubo latino (M03) en [0, 1]^6 y codificación de Gray	Hipercubo latino (M03) en [0, 1]^6	Hipercubo latino (M03) y redondeo a {0, …, 20}
2 · Evaluación	Aptitud directa (4)	Aptitud directa (4)	Aptitud directa (4)
3 · Selección	Torneo k = 2 con reemplazo	Torneo k = 2 con reemplazo	Torneo k = 2 con reemplazo
4 · Recombinación	Cruce de un punto	BLX-α, α = 0.3	Cruce uniforme por gen
5 · Mutación	Volteo de bit, 1/48	Gaussiana acotada, σ = 0.05, 1/6 por gen	Paso ±1, 1/6 por gen
6 · Reemplazo	Generacional con elitismo e = 2	Igual	Igual
7 · Paro	Presupuesto de 5 000 evaluaciones (4 990 usadas)	Igual	Igual

5.1 Etapa 1 · inicialización

Un hipercubo latino (McKay, Beckman y Conover, 1979) reparte N puntos en [0, 1]^6 de modo que cada estrato de cada dimensión contiene exactamente un punto. La población inicial es la misma en las tres codificaciones para una semilla dada.

$$x_{ij} = (\pi_j(i) + U_{ij}) / N , \quad \pi_j \text{ permutación aleatoria de } \{0, \ldots, N-1\} \quad (5)$$

5.2 Etapa 2 · evaluación

La evaluación es directa: la aptitud (4) de cada individuo. El problema no tiene restricciones, así que no se usa prioridad de factibilidad (M08), que sí aplica a los problemas con restricciones.

5.3 Etapa 3 · selección por torneo

Para cada padre se toman dos individuos al azar con reemplazo y gana el de mayor aptitud. Es el torneo M12 con k = 2, que no requiere aptitudes positivas ni proporcionales.

$$\text{padre} = \operatorname{argmax}_{j \in \{a, b\}} f(G_j) , \quad a, b \sim U\{1, \ldots, N\} \quad (6)$$

5.4 Etapa 4 · recombinación por codificación

G1, cruce de un punto. Con punto de corte $c \in \{1, \ldots, 47\}$ por pareja:

$$h^{(1)}_l = a_l \ (l < c) , \ b_l \ (l \geq c); \quad h^{(2)}_l = b_l \ (l < c) , \ a_l \ (l \geq c) \quad (7)$$

G2, BLX-α (Eshelman y Schaffer, 1993), con recorte a [0, 1]. Para $I_k = |a_k - b_k|$:

$$h_k \sim U[\ \min(a_k, b_k) - \alpha I_k , \ \max(a_k, b_k) + \alpha I_k\ ] \cap [0, 1], \quad \alpha = 0.3 \quad (8)$$

G3, cruce uniforme. Con máscara $m_k \sim \text{Bernoulli}(0.5)$:

$$h^{(1)}_k = m_k a_k + (1 - m_k) b_k , \quad h^{(2)}_k = (1 - m_k) a_k + m_k b_k \quad (9)$$

La pareja se cruza con probabilidad $p_c = 0.9$; si no, los hijos son copias de los padres.

5.5 Etapa 5 · mutación por codificación

G1: $b'_l = 1 - b_l$ con probabilidad 1/48 por bit (10a)

G2: $x'_k = \text{clip}_{[0,1]}(x_k + \sigma z)$, $z \sim N(0, 1)$, $\sigma = 0.05$, con probabilidad 1/6 por gen (10b)

G3: $g'_k = \text{clip}_{\{0..20\}}(g_k + s)$, $s \in \{-1, +1\}$ equiprobable, con probabilidad 1/6 por gen (10c)

La tasa 1/L para G1 es la tasa estándar de volteo de bit (Bäck, 1996). En las tres codificaciones, en promedio, cada individuo recibe un cambio por mutación (un bit en G1, un gen en G2 y G3). Sin embargo, el efecto fenotípico de un bit en G1 depende de su posición en el código de Gray, así que la intensidad de la mutación no es idéntica. Esta diferencia se discute como limitación en la sección 7.8.

5.6 Etapa 6 · reemplazo elitista

$$P^{(t+1)} = \text{Élite}_2(P^{(t)}) \cup H^{(t)}, \quad |H^{(t)}| = N - 2 = 58 \qquad (11)$$

5.7 Etapa 7 · paro por presupuesto

$$E_t = N + t\,(N - e), \quad \text{parar si } E_t + (N - e) > E_{max} \qquad (12)$$

Con N = 60, e = 2 y E_max = 5 000 se ejecutan 85 generaciones y se consumen 4 990 evaluaciones de aptitud en cada corrida. Las búsquedas aleatorias usan exactamente el mismo número de evaluaciones.

6. Código que apoya el cumplimiento del objetivo

El código consta de tres archivos. Los dos primeros son los scripts proporcionados, ejecutados sin modificación. El tercero es la aportación de este estudio.

Tabla 6. Archivos del código y su función

Archivo	Función	Entrada	Salida
synpuf_generator_v2.py	Genera reclamaciones, eventos y verdad de referencia	Parámetros y semilla	synpuf_event_log.csv, synpuf_claim_meta.csv, synpuf_truth.csv
cas_detector_v2.py	Calcula las seis señales por validación anidada y la ablación	Datos observables y etiquetas (sólo evaluación)	oof_predictions.csv, ablation_metrics.csv, audit.json
ga_codificacion.py	Algoritmo genético con tres codificaciones, controles, estadística y figuras	oof_predictions.csv	corridas.csv, pruebas_estadisticas.csv, figuras, manifiesto.json
construir_informe.py	Genera este documento con las cifras de los resultados	Carpetas de resultados	Documento DOCX

6.1 Cómo el código apoya el objetivo

- **Validación externa por proveedor** para el pliegue k, el algoritmo genético usa sólo los pliegues distintos de k para calcular la aptitud. El pliegue k se reserva para evaluar la solución final.
- **Controles con presupuesto idéntico** búsqueda aleatoria con 4 990 evaluaciones, pesos uniformes y la mejor señal individual elegida en entrenamiento.
- **Misma población inicial por semilla** las tres codificaciones parten del mismo hipercubo latino.
- **Pruebas automatizadas antes de cualquier corrida** seis pruebas verifican la biyección de Gray, la suma unitaria de los pesos, la igualdad de la aptitud vectorizada con average_precision_score, la forma de cruces y mutaciones y el rango de los fenotipos. Todas pasaron.

- **Inicio del experimento sólo tras las pruebas** el script no ejecuta el experimento si una prueba falla.

6.2 Errores detectados y corregidos durante el desarrollo

Durante la validación se detectó un error en la prueba estadística: el rango se calculaba en orden ascendente, de modo que el método elegido como mejor era el de peor desempeño. Se corrigió invirtiendo el orden del ranking y se recalcularon las pruebas sobre las mismas corridas, sin repetir el experimento. Las cifras de este informe provienen de la versión corregida.

6.3 Entorno de ejecución

Tabla 7. Versiones de las librerías utilizadas

Componente	Versión
Python	3.13.16
NumPy	2.5.3
pandas	3.0.5
SciPy	1.18.1
scikit-learn	1.9.1
XGBoost	3.4.1
PM4Py	2.7.23.11
Matplotlib	3.11.2

Las corridas se ejecutan con semillas fijas (detector: 2024; algoritmo genético: 11, 23, 37, 53 y 71). La reproducibilidad bit a bit no se verificó con una segunda ejecución independiente; las semillas fijas son la garantía disponible.

7. Resultados, interpretación y análisis

7.1 Desempeño del detector base

La Tabla 8 resume la ablación del detector con presupuesto de 5 % por pliegue. Las cifras son medias sobre los cinco pliegues de prueba.

Tabla 8. Ablación del detector, dificultad 1.0 (medias sobre pliegues)

Modelo	PR-AUC	ROC-AUC	Precisión 5 %	Recall 5 %	Recall monetario 5 %
Capa A (estructural)	0.415	0.730	0.443	0.384	0.329
Capa B (semántica)	0.617	0.921	0.565	0.508	0.740
Capa C (temporal)	0.262	0.733	0.281	0.248	0.222

Modelo	PR-AUC	ROC-AUC	Precisión 5 %	Recall 5 %	Recall monetario 5 %
Proceso completo, pesos uniformes	0.682	0.927	0.656	0.588	0.643
Proceso completo, pesos v1	0.713	0.943	0.680	0.611	0.687
Isolation Forest	0.740	0.948	0.669	0.609	0.730
XGBoost sin CAS	0.971	0.995	0.951	0.865	0.860
XGBoost con CAS	0.970	0.995	0.951	0.864	0.863
O-A-R, promedio de rangos	0.951	0.972	0.949	0.863	0.900

XGBoost con CAS alcanza una PR-AUC de 0.970 y una ROC-AUC de 0.995. La capa semántica es la señal más fuerte de las capas de proceso (PR-AUC 0.617). La capa temporal, por sí sola, tiene ROC-AUC 0.733 y PR-AUC 0.262, con la PR-AUC más baja de las capas de proceso. El ranking promedio de O-A-R (0.951) queda por debajo de XGBoost con CAS, lo que muestra que promediar señales más débiles diluye a la más fuerte. Esta observación motiva el problema de fusión que aborda el algoritmo genético.

El intervalo de confianza al 95 % por bootstrap agrupado por proveedor para la ROC-AUC de XGBoost con CAS es [0.992, 0.997]. La precisión de la regla OR, que marca como alerta cualquier señal, es de 0.694 con un recall de 0.938.

Por tipo de fraude, la regla OR detecta casi todos los fraudes de tipo F2 (fantasma, recall 1.000), F3 (desagregación, 0.996) y F1 (upcoding severo, 0.987). El upcoding leve (F5) es el más difícil: su recall es 0.121, probablemente porque el sobreprecio es pequeño frente a la variación legítima de tarifas; no se investigó con un análisis dedicado.

7.2 Resultados del algoritmo genético en prueba

La Tabla 9 reporta media y desviación estándar sobre los 25 bloques pliegue × semilla. Las líneas base se evalúan sobre los mismos pliegues y semillas.

Tabla 9. Métricas en el pliegue de prueba, dificultad 1.0 (media ± DE)

Método	PR-AUC	ROC-AUC	Precisión 5 %	Recall 5 %	Recall monetario 5 %
G1 binaria (Gray)	0.9712 ± 0.0078	0.9953 ± 0.0014	0.9509 ± 0.0777	0.8645 ± 0.0803	0.8654 ± 0.1067
G2 real	0.9712 ± 0.0078	0.9954 ± 0.0013	0.9509 ± 0.0777	0.8645 ± 0.0803	0.8655 ± 0.1066

Método	PR-AUC	ROC-AUC	Precisión 5 %	Recall 5 %	Recall monetario 5 %
G3 entera	0.9713 ± 0.0077	0.9954 ± 0.0012	0.9509 ± 0.0777	0.8645 ± 0.0803	0.8654 ± 0.1067
Búsqueda aleatoria	0.9687 ± 0.0092	0.9913 ± 0.0034	0.9508 ± 0.0777	0.8644 ± 0.0801	0.8862 ± 0.0879
Pesos uniformes	0.9534 ± 0.0139	0.9758 ± 0.0085	0.9489 ± 0.0769	0.8627 ± 0.0799	0.8966 ± 0.0810
Mejor señal individual	0.9703 ± 0.0074	0.9952 ± 0.0012	0.9509 ± 0.0777	0.8645 ± 0.0803	0.8637 ± 0.1077

Los 25 bloques son los cinco pliegues de prueba por cinco semillas. Precisión y recall al 5 % se saturan con este nivel de dificultad, por lo que la PR-AUC y el recall monetario son los que discriminan.

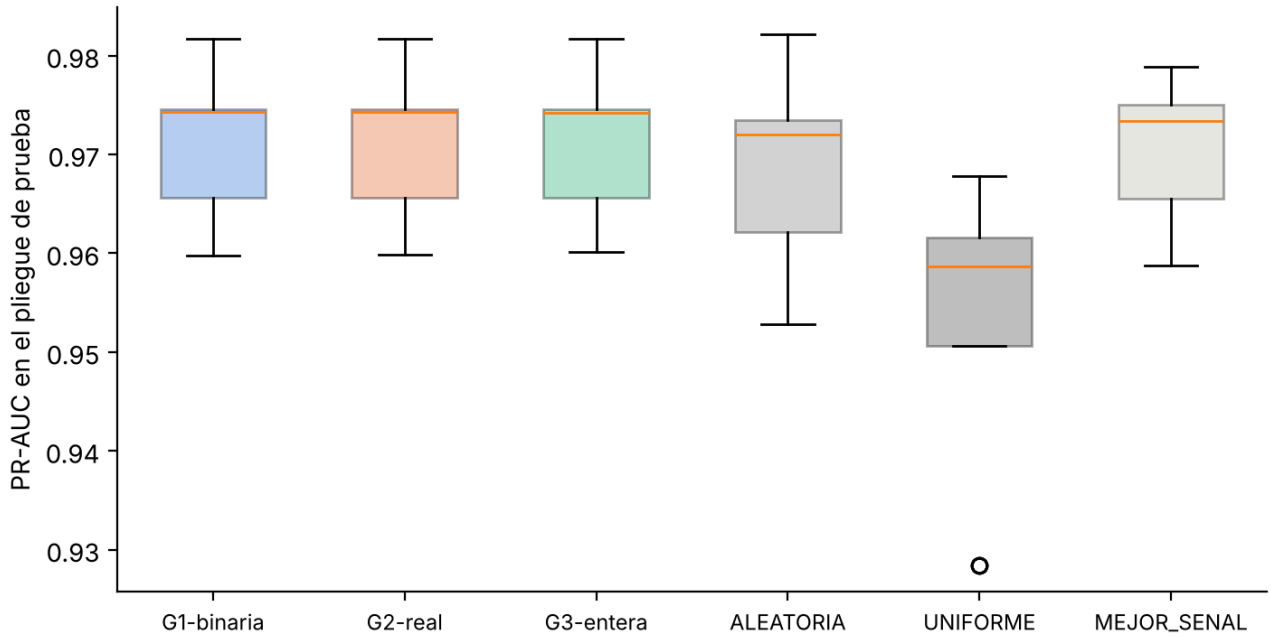


Figura 1. PR-AUC en prueba por método y codificación (dificultad 1.0). Cada caja agrupa los 25 bloques.

7.3 Pruebas estadísticas

Se aplica la prueba de Friedman (Friedman, 1937) sobre los 25 bloques y, para las comparaciones contra el mejor método, la prueba de rangos con signo de Wilcoxon (Wilcoxon, 1945) con ajuste de Holm (Holm, 1979). El nivel de significancia es $\alpha = 0.05$.

Tabla 10. Wilcoxon con ajuste de Holm contra el mejor método

Métrica	Comparación	p ajustado (Holm)
PR-AUC	G2 real vs G1 binaria (Gray)	0.3459
PR-AUC	G2 real vs G3 entera	0.5629
PR-AUC	G2 real vs Búsqueda aleatoria	< 0.0001
PR-AUC	G2 real vs Pesos uniformes	< 0.0001
PR-AUC	G2 real vs Mejor señal individual	0.0029

Friedman rechaza la igualdad de los seis métodos en PR-AUC ($\chi^2 = 88.2$, $p < 0.0001$). En precisión al 5 % hay muchos empates y la prueba ($\chi^2 = 69.7$, $p < 0.0001$) sólo separa a los pesos uniformes del resto.

Las tres codificaciones no difieren entre sí en calidad de prueba. La PR-AUC media es 0.9712 en G1, 0.9712 en G2 y 0.9713 en G3. Las diferencias son menores que la desviación entre pliegues. Esto confirma H3.

Frente a los controles, el algoritmo genético supera a la búsqueda aleatoria con el mismo presupuesto (PR-AUC 0.9712 frente a 0.9687) y a los pesos uniformes (0.9534). Confirma H1. La mejora sobre la mejor señal individual es estadísticamente significativa, pero de magnitud muy pequeña (PR-AUC 0.9703 frente a 0.9712); confirma H4 en significancia y matiza su importancia práctica.

7.4 Convergencia, diversidad y costo

Tabla 11. Diversidad final y costo por corrida (dificultad 1.0)

Codificación	Fracción de genotipos únicos al final	DE	Segundos por corrida	Aptitud de entrenamiento
G1 binaria (Gray)	0.946	0.038	31.7	0.9714
G2 real	0.981	0.022	31.7	0.9714
G3 entera	0.474	0.101	31.2	0.9714

Valores de codificación por fracción de genotipos únicos al final por segundos por corrida y aptitud de entrenamiento identificado.

La diversidad final confirma H2. G2 conserva casi toda su población distinta (0.98 de genotipos únicos), G1 también (0.95) y G3 pierde diversidad (0.47). La causa más probable es el espacio finito de G3: con 21 valores por gen, la población tiende a converger a un número reducido de configuraciones; el escape de las mutaciones no se midió directamente. Esa diversidad no se traduce en calidad: las tres codificaciones llegan al mismo óptimo.

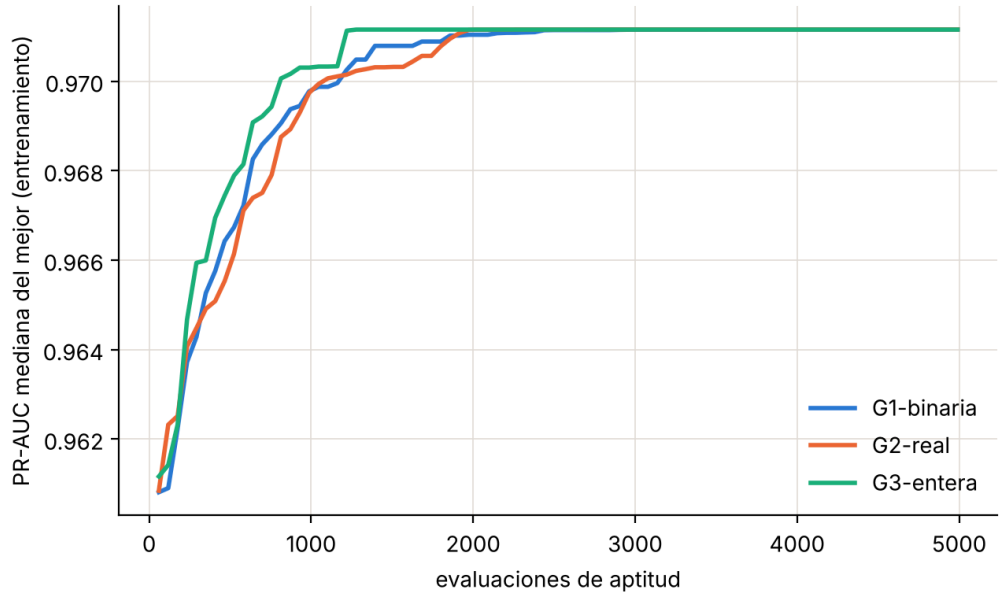


Figura 2. Convergencia de la mejor aptitud de entrenamiento frente a las evaluaciones (mediana de 25 corridas).

El costo por corrida es prácticamente idéntico entre codificaciones (unos 32 s con dos procesos en paralelo), porque la evaluación domina el tiempo y es la misma en las tres. En convergencia, la mediana de la mejor aptitud de entrenamiento entra en 0.0005 del valor final en la evaluación 1,394 (G1), 1,800 (G2) y 1,220 (G3) (Figura 2).

7.5 Pesos aprendidos

Tabla 12. Peso medio de cada señal en la solución final (dificultad 1.0)

Señal	G1 binaria	G2 real	G3 entera	Mejor señal (proporción de pliegues)
Estructural (A)	0.000	0.000	0.000	0.000
Semántica (B)	0.000	0.000	0.000	0.000
Temporal (C)	0.001	0.001	0.000	0.000
XGBoost con CAS	0.431	0.434	0.439	0.200
Isolation Forest	0.000	0.000	0.000	0.000
XGBoost sin CAS	0.568	0.566	0.561	0.800

Pesos medios de cada señal identificada en la solución final con valor de dificultad = 1.0

Las tres codificaciones asignan prácticamente todo el peso a los dos XGBoost: 0.999 en G2. Las señales estructural, semántica, temporal e Isolation Forest reciben peso casi nulo. Esto indica que, dado XGBoost, las señales de proceso no aportan información adicional en la fusión lineal, al menos con estos datos. La fusión aprendida es, en la práctica, un promedio de dos modelos XGBoost con distinta variante de variables, y su ventaja sobre XGBoost sólo es marginal.

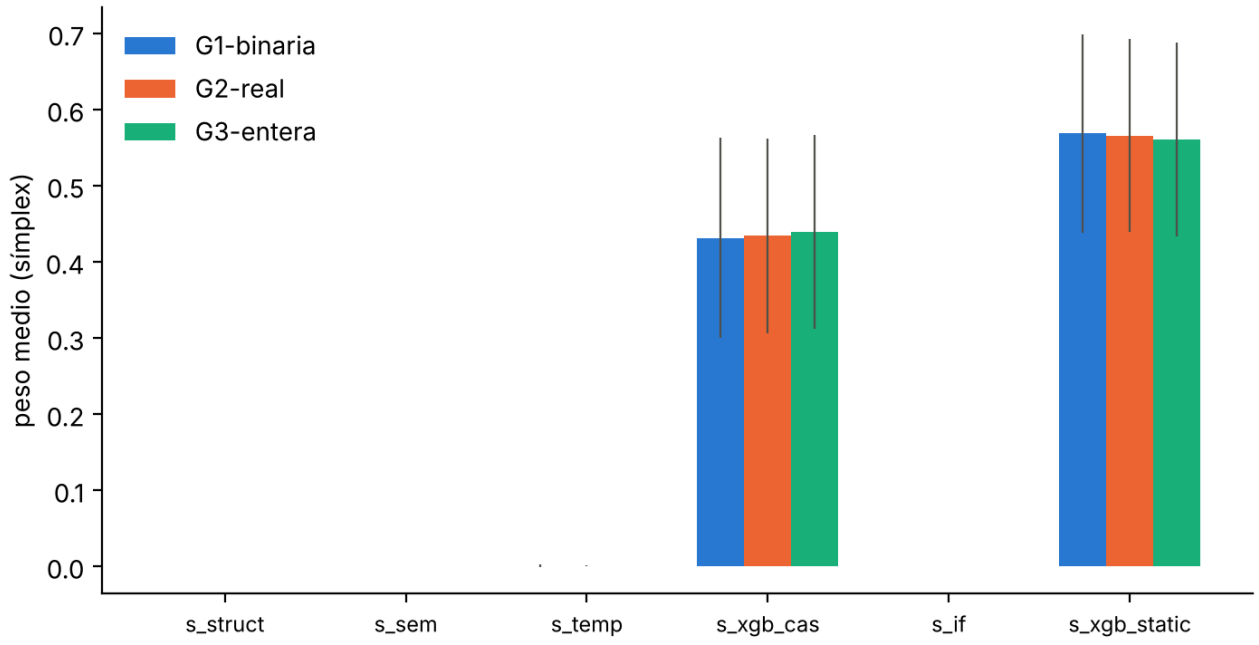


Figura 3. Peso medio por señal y codificación (barras: DE entre corridas).

7.6 Prueba de estrés con dificultad 0.25

Tabla 13. Resultados del algoritmo genético con dificultad 0.25 (media ± DE)

Método	PR-AUC (prueba)	Recall monetario 5 %
G1 binaria (Gray)	0.9243 ± 0.0209	0.8578 ± 0.0950
G2 real	0.9243 ± 0.0209	0.8578 ± 0.0950
G3 entera	0.9239 ± 0.0213	0.8576 ± 0.0949
Búsqueda aleatoria	0.9240 ± 0.0212	0.8560 ± 0.0944
Pesos uniformes	0.9067 ± 0.0209	0.8790 ± 0.0833
Mejor señal individual	0.9235 ± 0.0219	0.8404 ± 0.0930

Con dificultad 0.25 la PR-AUC cae en todos los métodos. La ventaja de la fusión sobre la mejor señal individual es de +0.0008 en PR-AUC, frente a +0.0009 con dificultad 1.0. La hipótesis H5 no se sostiene: la ventaja de la fusión no aumenta al debilitarse la señal.

Con dificultad 0.25, la prueba de Friedman sobre PR-AUC rechaza la igualdad de los seis métodos ($\chi^2 = 58.4$, $p < 0.0001$). El mejor rango corresponde a G1 binaria (Gray). G2 no difiere de G1 (p de Holm = 0.9762), mientras que G3 es significativamente inferior (p de Holm < 0.0001). Esta diferencia coincide con la menor diversidad final de G3 (0.68, frente a 0.94 en G1 y 0.98 en G2).

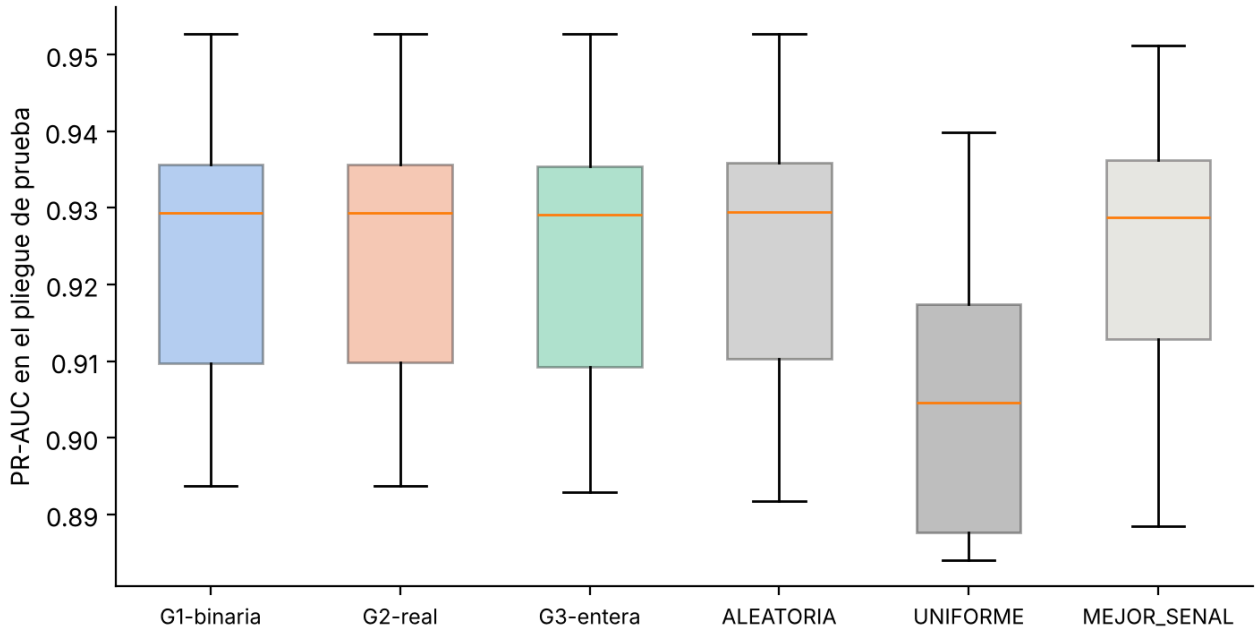


Figura 4. PR-AUC en prueba con dificultad 0.25 por método y codificación.

Tabla 14. PR-AUC del detector base según la dificultad de los datos

Dificultad	XGBoost con CAS	Proceso completo, pesos uniformes	O-A-R promedio
1.0	0.970	0.682	0.951
0.5	0.952	0.560	0.932
0.25	0.922	0.443	0.900

PR-AUC por rango de dificultad en comparación con XG Boost por proceso completo y O-A-R promedio identificado.

7.7 Discusión de los resultados

- **Lo que importa es la fusión, no la codificación** las tres codificaciones encuentran la misma solución. La elección de representación afecta a la diversidad, no a la calidad, en este problema.
- **La fusión aprendida es un promedio de XGBoost** el aporte de las capas de proceso desaparece al fusionar con XGBoost. Esto no prueba que sean inútiles en otro contexto: en datos reales, las reglas de proceso pueden capturar fraudes que los clasificadores no ven.
- **El presupuesto de 5 % satura las métricas de precisión y recall** con dificultad 1.0 casi todos los métodos alcanzan precisión de 0.95. La PR-AUC y el recall monetario son las métricas que discriminan, y el recall monetario favorece a los pesos uniformes. Una explicación plausible es que los pesos uniformes conservan peso en la capa semántica, que incorpora la razón de monto; esta explicación no se verificó con un análisis dedicado.
- **El riesgo de sobreajuste bajo** la aptitud se calcula sólo con pliegues de entrenamiento y la validación es por proveedor. La aptitud media de entrenamiento de las corridas genéticas es 0.9714 y su PR-AUC media en prueba es 0.9712; la diferencia es pequeña y sugiere poco sobreajuste.

7.8 Limitaciones

- **Efecto fenotípico de la mutación no equiparado** en promedio, las tres codificaciones aplican un cambio por individuo, pero el efecto de un bit en G1 depende de su posición en el código de Gray. Una comparación con magnitud fenotípica equiparada podría alterar los resultados.
- **Un solo presupuesto y una sola población** N = 60 y E = 4 990 se fijaron a priori; no se explora la sensibilidad a estos valores.
- **Dificultad: se exploran** tres niveles de dificultad para el detector (1.0, 0.5 y 0.25) y dos para el algoritmo genético, no un barrido continuo.

8. Conclusiones

El estudio aborda la detección de fraude en reclamaciones médicas mediante un detector en tres capas (estructural, semántica y temporal) cuyas señales de salida se

fusionan con un algoritmo genético. Los resultados sostienen que la fusión supera de forma estadísticamente significativa a los controles de igual presupuesto, pero no que mejore de manera sustancial a la mejor señal individual.

La principal fortaleza del diseño es su rigor en la evaluación. La validación se realiza con pliegues agrupados por proveedor, de modo que ningún proveedor aparece en entrenamiento y prueba a la vez, y las etiquetas que guían la aptitud del algoritmo genético provienen únicamente de los pliegues de entrenamiento. Los controles, que incluyen búsqueda aleatoria, pesos uniformes y la mejor señal individual, operan con un presupuesto de evaluaciones equivalente, y las comparaciones se respaldan con pruebas de Friedman y de Wilcoxon con ajuste de Holm. Además, la incorporación de niveles de dificultad 0.5 y 0.25 permitió comprobar que las conclusiones no dependen de un conjunto de datos casi separable, como ocurre en el nivel base, donde el XGBoost con señales CAS alcanza un ROC-AUC cercano a 0.995.

Entre las limitaciones, la más relevante es que el conjunto de datos es sintético. El generador se diseñó con supuestos explícitos, por lo que los resultados describen el comportamiento bajo esos supuestos y no pueden extrapolarse directamente a reclamaciones reales. En segundo lugar, la ganancia de la fusión sobre la mejor señal individual es mínima (+0.0009 en dificultad 1.0 y +0.0008 en dificultad 0.25), de modo que la hipótesis H5 no se confirma. En tercer lugar, existe un riesgo de fuga por apilamiento, dado que las señales del detector son entradas del algoritmo genético; el documento lo reconoce como limitación y no lo elimina por completo. Asimismo, el tipo de fraude F5 (sobrefacturación) se recupera solo en el 12 % de los casos con la regla OR, y dicha regla presenta una precisión de 0.694 con un recall de 0.938. Por último, no se verificó la reproducibilidad bit a bit de los resultados, y el enlace al repositorio es provisional.

Desde el punto de vista del código, la contribución principal es ga_codificacion.py, que implementa la fusión de señales sobre el símplex de pesos con tres codificaciones (binaria Gray, real con cruce BLX-α y entera), calcula la aptitud como PR-AUC mediante una implementación verificada contra scikit-learn, y ejecuta auto verificaciones antes de cada experimento. Durante la revisión se corrigió un error en el cálculo de rangos de Friedman que asignaba el peor rango al mejor método.

En cuanto a los resultados sustantivos, las tres codificaciones convergen a una misma combinación de pesos, aproximadamente 0.43 para el XGBoost con señales CAS y 0.57 para el XGBoost estático, y no presentan diferencias significativas en PR-AUC con dificultad 1.0. Con dificultad 0.25, la codificación entera (G3) resulta significativamente inferior a la real (G2), y su diversidad final es menor. La conclusión que se desprende es que, en este problema, la fusión proporciona sobre todo robustez frente a la elección manual de pesos, mientras que la señal de XGBoost con características CAS sigue siendo la dominante.

9. Referencias bibliográficas

Bäck, T. (1996). Evolutionary algorithms in theory and practice: Evolution strategies, evolutionary programming, genetic algorithms. Oxford University Press.

Berti, A., van Zelst, S. J., y van der Aalst, W. M. P. (2019). Process mining for Python (PM4Py): Bridging the gap between process- and data science. En Proceedings of the ICPM 2019 Demo Track (CEUR Workshop Proceedings).

Bauder, R. A., y Khoshgoftaar, T. M. (2017). Medicare fraud detection using machine learning methods. En 16th IEEE International Conference on Machine Learning and Applications (ICMLA).

Centers for Medicare & Medicaid Services. (s. f.). Medicare claims synthetic public use files (SynPUF). CMS. https://www.cms.gov/

Chen, T., y Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785–794).

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. Journal of Machine Learning Research, 7, 1–30.

Eshelman, L. J., y Schaffer, J. D. (1993). Real-coded genetic algorithms and interval-schemata. En Foundations of Genetic Algorithms 2 (pp. 187–202). Morgan Kaufmann.

Friedman, M. (1937). The use of ranks to avoid the assumption of normality implicit in the analysis of variance. Journal of the American Statistical Association, 32(200), 675–701.

Goldberg, D. E. (1989). Genetic algorithms in search, optimization, and machine learning. Addison-Wesley.

Holland, J. H. (1975). Adaptation in natural and artificial systems. University of Michigan Press.

Holm, S. (1979). A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics, 6(2), 65–70.

Leemans, S. J. J., Fahland, D., y van der Aalst, W. M. P. (2013). Discovering block-structured process models from event logs: A constructive approach. En Application and Theory of Petri Nets and Concurrency (PETRI NETS 2013), LNCS 7927, 311–329.

Liu, F. T., Ting, K. M., y Zhou, Z.-H. (2008). Isolation forest. En Proceedings of the 8th IEEE International Conference on Data Mining (ICDM) (pp. 413–422).

McKay, M. D., Beckman, R. J., y Conover, W. J. (1979). A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. Technometrics, 21(2), 239–245.

Saito, T., y Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLoS ONE, 10(3), e0118432.

van der Aalst, W. M. P. (2016). Process mining: Data science in action (2.ª ed.). Springer.

Wilcoxon, F. (1945). Individual comparisons by ranking methods. Biometrics Bulletin, 1(6), 80–83.

Wolpert, D. H. (1992). Stacked generalization. Neural Networks, 5(2), 241–259.